Chapter 5 — Objects, Math, Date and JSON

Almost everything interesting in JavaScript is an **object**: a user profile, an HTTP response (a web server's reply — System Design Fundamentals), a config file, a DOM element (one piece of a web page as the browser exposes it), even arrays and functions. This chapter covers the object toolkit — creating, reading, changing, copying, merging and locking objects — then three built-ins you'll use constantly: `Math`, `Date` and `JSON`.

What you'll learn: the object toolkit (§5.1–5.8), the built-in objects (§5.9), `Math` (§5.10), `Date` and **Temporal** (§5.11), and JSON (§5.12).

Before you start: variables, types, operators (Chapter 2); functions and arrows (Chapter 4). Arrays: Chapter 7; `Map` / `Set` (often better lookup tables): Chapter 8.

Code is in `code/`. Every file is an ES module (`.mjs`, the `import` / `export` system — Chapter 10), so it runs in **strict mode** (silent failures become errors). Outputs are real Node 24 output.

5.1 What an object is

What it is

An **object** is a collection of **properties**: each a **key** (normally a string) paired with a **value** (any value — number, string, object, function). A property whose value is a function is a **method**. A car has properties (colour, brand) and methods (drive, brake). Unlike a **primitive** (string, number, boolean, `null`, `undefined`, bigint, symbol) — a single unchangeable value — an object groups related values and behaviour under one name.

Why it matters

Real data is structured: a user is a name, email, roles, signup date. Objects keep those together and attach behaviour. JSON (§5.12), the format of nearly every web API (a service other programs call over the internet), is a text version of objects and arrays.

How it works — creating objects with a literal

An **object literal** — curly braces with comma-separated `key: value` pairs — is the most common way:

```
const car = {
  color: 'red',
  brand: 'Toyota',
  model: 'Corolla',
  drive() {                       // a method
    console.log('The car is moving');
  },
};
```

- `{ ... }` creates a new object each time the line runs.
- `drive() { ... }` is **method shorthand** (ES2015) for `drive: function () { ... }`.
- A trailing comma is allowed and keeps diffs (version-control "what changed" views) clean.

Alternatives — `new Object()` (never needed), `Object.create(proto)` (§5.6), constructors and classes (Chapter 9) — but literals cover ~90% of code.

5.2 Reading, adding, updating and deleting properties

Dot notation vs bracket notation

	Dot notation `obj.key`	**Bracket notation** `obj['key']` / `obj[expr]`
Key must be a valid identifier?	Yes (letters, digits, `_`, `$`; cannot start with a digit; no spaces or dashes)	No — any string (or symbol, a unique key type from Chapter 8) works
Key known while writing the code?	Yes — the name is typed literally	Not necessary — the key can come from a variable or expression
Readability	Cleaner; the default choice	A little noisier
Typical use	`user.name`, `config.port`	`headers['content-type']`, `row[columnName]`, `scores[2024]`

Use dot notation unless you can't — special characters, or a key known only at **run time** (while the program runs).

Computed keys and shorthand properties

- **Computed keys** (ES2015) — `[expression]: value` uses the expression's *result* as the key.
- **Shorthand properties** — `{ name, age }` means `{ name: name, age: age }`.

`code/01_object_basics.mjs` (first part):

```
const field = 'price';

const laptop = {
  brand: 'Lenovo',            // plain property
  'model name': 'ThinkPad',   // key that is not a valid identifier -> must be quoted
  [field]: 1299,              // computed key: the value of `field` becomes the key
  [`${field}WithTax`]: 1299 * 1.2,
  open() {                    // method shorthand
    return `${this.brand} ${this['model name']} is open`;
  },
};

console.log(laptop.brand);                // known, valid identifier
console.log(laptop['model name']);        // spaces -> brackets required
const key = 'price';
console.log(laptop[key], laptop.priceWithTax);
console.log(laptop.color);                // missing property -> undefined, not an error

const name = 'Ada', age = 36;
const user = { name, age };
console.log(user);
```

Output:

```
Lenovo
ThinkPad
1299 1558.8
undefined
{ name: 'Ada', age: 36 }
```

A missing property reads as `undefined`; reading a property *of* `undefined` (`laptop.color.length`) throws `TypeError` — §5.8's `?.` avoids it.

Adding, updating and deleting

Objects are **mutable**: assigning to an existing key updates it, to a new key adds it; `delete obj.key` removes it.

```
laptop.brand = 'Dell';          // update (dot)
laptop['ram'] = '16GB';         // add (bracket)
delete laptop.priceWithTax;     // remove
console.log(laptop.open(), '|', Object.keys(laptop).join(','));
```

Output:

```
Dell ThinkPad is open | brand,model name,price,open,ram
```

Does the object have this key?

```
console.log('ram' in laptop, Object.hasOwn(laptop, 'ram'), laptop.ram !== undefined);
const withUndefined = { x: undefined };
console.log('x' in withUndefined, withUndefined.x !== undefined);
```

Output:

```
true true true
true false
```

Check	Looks at inherited (prototype) keys?	Works when the value is `undefined`?
`'key' in obj`	Yes (so `'toString' in {}` is `true`)	Yes
`Object.hasOwn(obj, 'key')` (ES2022)	No — own properties only	Yes
`obj.hasOwnProperty('key')`	No	Yes, but fails on objects without a prototype; prefer `Object.hasOwn`
`obj.key !== undefined`	Yes	**No** — a key explicitly set to `undefined` looks missing

(*Inherited* keys come from the object's **prototype** — the fallback object checked when a key isn't found on the object itself; §5.6.)

Key order

Order is fully specified since ES2020 for `Object.keys`, `for...in`, `JSON.stringify` etc.:

1. **Integer-like keys** (`'0'`, `'42'`) ascending;
2. other **string keys** in insertion order;
3. **symbol keys** in insertion order (only reported by symbol-aware functions).

```
const ordered = { b: 1, 2: 'two', a: 2, 1: 'one', [Symbol('s')]: 3 };
console.log(Object.keys(ordered));
console.log(Reflect.ownKeys(ordered));
```

Output:

```
[ '1', '2', 'b', 'a' ]
[ '1', '2', 'b', 'a', Symbol(s) ]
```

Deterministic, but not pure insertion order for number-like keys. For insertion order always (or non-string keys) use a `Map` (Chapter 8).

Objects are references

A variable holds a **reference** to the object; copying the variable copies the reference.

```
const a = { n: 1 };
const b = a;           // same object, second name
b.n = 99;
console.log(a.n, a === b, { n: 1 } === { n: 1 });

const cfg = { debug: false };
cfg.debug = true;      // allowed
console.log(cfg);
try { cfg = {}; } catch (e) { console.log(e.constructor.name + ':', e.message); }
```

Output:

```
99 true false
{ debug: true }
TypeError: Assignment to constant variable.
```

- `===` on objects means "same object?", not "same contents?".
- `const` freezes the **binding**, not the object — use `Object.freeze` (§5.6).

Common mistakes

- `obj.field` when the key is in variable `field` — looks for a property literally named "field"; use `obj[field]`.
- Expecting insertion order for number-like keys.
- Believing `const` makes an object immutable.
- `===` for "equal contents" — compare fields, or deliberately serialize both (e.g. to JSON) and compare strings.

Self-check

1. Read a property named `first-name`? → `obj['first-name']`.
2. `const k = 'x'; ({ [k]: 1 })`? → `{ x: 1 }`.
3. `'toString' in {}` vs `Object.hasOwn({}, 'toString')`? → `true` vs `false`.
4. `Object.keys({ z: 1, 10: 1, 2: 1 })`? → `['2', '10', 'z']`.

5.3 Methods and `this`

What it is

Calling `obj.method()` sets **`this`** inside the method to `obj` (the object left of the dot), so a method can read the object's other properties.

```
const person = {
  firstName: 'Grace',
  lastName: 'Hopper',
  greet() {
    console.log(`Hi, I'm ${this.firstName} ${this.lastName}`);
  },
};
person.greet();   // Hi, I'm Grace Hopper
```

Brackets work too: `person['greet']()`.

Common mistakes

- **Arrow functions as methods** — arrows borrow `this` from surrounding code, so they don't see the object:

```
const counter = {
  count: 0,
  incArrow: () => typeof this,          // `this` is the module's this (undefined)
  inc() { this.count++; return this.count; },
};
console.log(counter.incArrow(), counter.inc());
```

Output (from `code/05_getters_setters_optional.mjs`):

```
undefined 1
```

- **Detaching a method** — `const f = person.greet; f();` loses `this` (`undefined` in strict mode). Full rules (default, implicit, explicit `call`/`apply`/`bind`, `new`): Chapter 9, section 9.1.

Returning `this` for chaining

Returning `this` enables chained calls — the style of jQuery, query builders (step-by-step database query libraries) and test libraries:

```
deposit(amount) { this._balance += amount; return this; }
// account.deposit(10).deposit(5)
```

5.4 Spread, rest and copying objects

What it is

- **Spread** `{ ...source }` (ES2018) copies `source`'s *own* (not inherited), *enumerable* (visible to loops and listing methods) properties into a new literal.
- **Rest** in destructuring, `const { a, ...others } = obj`, gathers the un-named properties into a new object.

How it works

Properties copy left to right; for duplicate keys the **later one wins** (overrides on top of defaults); only the top level is copied — nested objects are shared (a **shallow copy**).

From `code/02_object_static_methods.mjs`:

```
const defaults = { theme: 'light', lang: 'en', fontSize: 14 };
const prefs = { theme: 'dark' };
const merged = Object.assign({}, defaults, prefs); // later sources win
console.log(merged);

console.log({ ...defaults, ...prefs, fontSize: 16 });

const { password, ...safeUser } = { id: 7, name: 'Lin', password: 'x' };
console.log(safeUser);

const original = { name: 'Kit', address: { city: 'Pune' } };
const shallow = { ...original };
const deep = structuredClone(original);
shallow.address.city = 'Delhi';
console.log(original.address.city, deep.address.city);
```

Output:

```
{ theme: 'dark', lang: 'en', fontSize: 14 }
{ theme: 'dark', lang: 'en', fontSize: 16 }
{ id: 7, name: 'Lin' }
Delhi Pune
```

The last line is the classic bug: `shallow` and `original` share the inner address object. **`structuredClone(value)`** (global in browsers and Node 17+) makes a **deep copy** — nested objects, arrays,

`Date`, `Map`, `Set`, typed arrays, even circular references (objects pointing back to themselves). It can't copy functions, DOM nodes or class identity (class instances come back as plain objects).

Trade-offs: ways to copy or merge

(*Getter/setter*: a property that runs a function when read/written — §5.7.)

Technique	Depth	Mutates target?	Copies getters as getters?	Notes
`{ ...a, ...b }`	Shallow	No (new object)	No — getters are *called*, values copied	Most common; readable
`Object.assign(target, a, b)`	Shallow	**Yes** — writes into `target`	No — getters called, setters on target triggered	Use `Object.assign({}, ...)` for a fresh object
`structuredClone(obj)`	Deep	No	No	Handles Date/Map/Set/cycles; throws on functions
`JSON.parse(JSON.stringify(obj))`	Deep	No	No	Lossy: Dates → strings, drops `undefined`/functions, fails on cycles and BigInt (§5.12)

Real-world examples

- Redux reducers and React state updates: `return { ...state, loading: false }` — new object, no mutation.
- Layered config: `{ ...defaultConfig, ...fileConfig, ...envConfig }`.
- Stripping sensitive fields: `const { password, ...safe } = user`.

In the interview

> **30-second answer:** "Spread copies own enumerable properties into a new literal; later keys override. It's shallow — nested objects are shared. For deep copies I use `structuredClone` (Dates, Maps, Sets, cycles); the JSON round-trip is lossy."

- *`Object.assign` vs spread?* `assign` mutates its first argument and triggers its setters; spread always makes a new object.
- *Deep-merge two configs?* Recurse: if both values are plain objects merge them, else take the right-hand value — or lodash `merge` (a popular utility library).

Self-check

1. `{ ...{ a: 1 }, a: 2 }` → `{ a: 2 }`.
2. After `const c = { ...o }`, does `c.list.push(1)` change `o.list`? → Yes (shallow).
3. `structuredClone` of a method? → Throws `DataCloneError`.

5.5 Destructuring objects

What it is

Destructuring (ES2015) unpacks properties into variables: `const { a, b } = obj` ≈ `const a = obj.a, b = obj.b`.

How it works

From `code/03_destructuring.mjs`:

```js
const order = {
  id: 'A-17',
  customer: { name: 'Mira', address: { city: 'Lyon' } },
  items: 3,
};

const { id, items } = order;                      // basic
const { id: orderId } = order;                    // rename: take `id`, call it orderId
const { status = 'pending' } = order;             // default when the value is undefined
const { customer: { name, address: { city } } } = order; // nested
console.log(id, items, orderId, status, name, city);

const { a = 1, b = 2 } = { a: undefined, b: null };
console.log(a, b);

function createUser({ name, role = 'viewer', active = true } = {}) {
  return `${name ?? 'anonymous'} / ${role} / ${active}`;
}
console.log(createUser({ name: 'Jo', role: 'admin' }));
console.log(createUser());

let x, y;
({ x, y } = { x: 10, y: 20 });
console.log(x, y);

try { const { q } = null; } catch (e) { console.log(e.constructor.name); }
```

Output:

```
A-17 3 A-17 pending Mira Lyon
1 null
Jo / admin / true
anonymous / viewer / true
10 20
TypeError
```

- `{ id: orderId }` — "take `id`, store in `orderId`": the **right** of the colon is the new variable.
- Defaults apply only for `undefined`, **not** `null`.
- In `customer: { name }`, `customer` is only a path — no variable is created.
- **Parameter destructuring** gives "named arguments": order-independent options with defaults; trailing `= {}` makes a no-argument call safe.
- Assigning to *existing* variables needs parentheses (else `{` starts a block).
- Destructuring `null`/`undefined` throws `TypeError`.

Array destructuring (`const [first, second] = arr`): Chapter 7.

Self-check

1. `const { a: { b } } = { a: { b: 5 } }` — variables? → Only `b` (5).
2. `const { n = 10 } = { n: 0 }` → `n` is `0`.
3. Why `function f({ x } = {})`? → So `f()` doesn't throw.

5.6 `Object.*` static methods, descriptors and locking

What it is

`Object` holds **static methods** called as `Object.method(obj)`:

Group	Methods	What they do
Listing	`Object.keys(o)`, `Object.values(o)`, `Object.entries(o)`, `Object.fromEntries(pairs)`	Own enumerable string-keyed names / values / `[key, value]` pairs; `fromEntries` rebuilds an object from pairs
Copying & creating	`Object.assign(target, ...sources)`, `Object.create(proto, descriptors?)`	Copy properties into a target; make a new object with a chosen prototype
Reflection (code inspecting the structure of objects at run time)	`Object.getOwnPropertyNames(o)` (includes non-enumerable string keys), `Object.getOwnPropertySymbols(o)`, `Object.getOwnPropertyDescriptor(o, k)`, `Object.getOwnPropertyDescriptors(o)`, `Object.getPrototypeOf(o)`, `Object.setPrototypeOf(o, p)`, `Object.hasOwn(o, k)`, `Object.is(a, b)`	Inspect keys, flags and the prototype; compare values
Defining & locking	`Object.defineProperty(o, k, desc)`, `Object.defineProperties(o, descs)`, `Object.preventExtensions`, `Object.seal`, `Object.freeze`, and the tests `isExtensible`, `isSealed`, `isFrozen`	Control exactly how a property behaves; stop an object from changing

Listing: keys, values, entries, fromEntries

From `code/02_object_static_methods.mjs`:

```
const stock = { apples: 5, pears: 0, plums: 12 };

console.log(Object.keys(stock));
console.log(Object.values(stock));
console.log(Object.entries(stock));

const inStock = Object.fromEntries(
  Object.entries(stock).filter(([, qty]) => qty > 0).map(([k, qty]) => [k.toUpperCase(), qty]),
);
console.log(inStock);

for (const [fruit, qty] of Object.entries(stock)) console.log(`${fruit.padEnd(7)}${qty}`);
```

Output:

```
[ 'apples', 'pears', 'plums' ]
[ 5, 0, 12 ]
[ [ 'apples', 5 ], [ 'pears', 0 ], [ 'plums', 12 ] ]
{ APPLES: 5, PLUMS: 12 }
apples 5
pears  0
plums  12
```

entries → array methods → `fromEntries` is the idiomatic way to map/filter an object (objects have no `.map`). The listing methods return only **own, enumerable, string-keyed** properties in §5.2's order — unlike `for...in`, which also walks inherited keys (Chapter 3).

`Object.create` and `Object.is`

```
const animal = { speak() { return `${this.name} makes a sound`; } };
const dog = Object.create(animal);
dog.name = 'Rex';
console.log(dog.speak(), Object.getPrototypeOf(dog) === animal, Object.keys(dog));
const dict = Object.create(null);    // no prototype at all: a "pure" dictionary
console.log('toString' in dict, 'toString' in {});

console.log(NaN === NaN, Object.is(NaN, NaN), 0 === -0, Object.is(0, -0));
```

Output:

```
Rex makes a sound true [ 'name' ]
false true
false true true false
```

- `Object.create(animal)` makes an empty object whose **prototype** is `animal`; `speak` is found there (Chapter 9).
- `Object.create(null)` has *no* prototype — no inherited `toString`, `constructor`, `__proto__` — a safe dictionary for user-supplied keys.
- `Object.is` = `===` except `NaN` equals itself and `+0` / `-0` differ. `Map` / `Set` use nearly the same rule (but treat `+0` and `-0` as equal).

Property descriptors

Every property has hidden **attributes**, collected in a **property descriptor**:

Attribute	Meaning	Default for normal assignment	Default via `defineProperty`
`value`	The stored value	—	`undefined`
`writable`	Can the value be changed?	`true`	`false`
`enumerable`	Shows up in `Object.keys`, `for...in`, spread, `JSON.stringify`?	`true`	`false`
`configurable`	Can the property be deleted or its attributes changed?	`true`	`false`
`get` / `set`	Accessor functions (instead of `value` / `writable`) — §5.7	—	`undefined`

From `code/04_descriptors_freeze.mjs`:

```
const tryIt = (label, fn) => {
  try { fn(); console.log(label, '-> ok'); } catch (e) { console.log(label, '->', e.constructor.name); }
};

const product = { name: 'Pen' };
console.log(Object.getOwnPropertyDescriptor(product, 'name'));

Object.defineProperty(product, 'sku', { value: 'P-001' });
console.log(Object.getOwnPropertyDescriptor(product, 'sku'));
console.log(Object.keys(product), Object.getOwnPropertyNames(product), product.sku);
tryIt('write read-only sku', () => { product.sku = 'X'; });
tryIt('delete non-configurable sku', () => { delete product.sku; });

Object.defineProperties(product, {
  price: { value: 2.5, writable: true, enumerable: true, configurable: true },
});
console.log(JSON.stringify(product)); // non-enumerable sku is skipped

const secret = Symbol('secret');
product[secret] = 42;
console.log(Object.keys(product), Object.getOwnPropertySymbols(product));
```

Output:

```
{ value: 'Pen', writable: true, enumerable: true, configurable: true }
{
  value: 'P-001',
  writable: false,
  enumerable: false,
  configurable: false
}
[ 'name' ] [ 'name', 'sku' ] P-001
write read-only sku -> TypeError
delete non-configurable sku -> TypeError
{"name":"Pen","price":2.5}
[ 'name', 'price' ] [ Symbol(secret) ]
```

Plain assignment sets every flag `true`; `defineProperty` sets every omitted flag `false`. Writing a read-only property throws in **strict mode** but is silently ignored in **sloppy mode** ("why didn't my change stick?").

Locking an object: preventExtensions, seal, freeze

```
const ext = { a: 1 };  Object.preventExtensions(ext);
const sealed = { a: 1 }; Object.seal(sealed);
const frozen = { a: 1, inner: { b: 1 } }; Object.freeze(frozen);
tryIt('seal: delete             ', () => { delete sealed.a; });
tryIt('freeze: nested change    ', () => { frozen.inner.b = 2; }); // freeze is shallow!
// ... the file tries add / change / delete on each of the three objects
console.log(Object.isExtensible(ext), Object.isSealed(sealed), Object.isFrozen(frozen),
Object.isFrozen(frozen.inner));
```

Output (full run — one `tryIt` per operation):

```
preventExtensions: add    -> TypeError
preventExtensions: change -> ok
preventExtensions: delete -> ok
seal: add                 -> TypeError
seal: change              -> ok
seal: delete              -> TypeError
freeze: change            -> TypeError
freeze: nested change     -> ok
false true true false
```

Level	Add new keys	Delete keys	Change values	Implemented as
`Object.preventExtensions`	✗	✓	✓	object marked non-extensible
`Object.seal`	✗	✗	✓	+ every property `configurable: false`
`Object.freeze`	✗	✗	✗	+ every data property `writable: false`

All three are **shallow**. A recursive helper freezes everything:

```
function deepFreeze(obj) {
  for (const value of Object.values(obj)) {
    if (value && typeof value === 'object') deepFreeze(value);
  }
  return Object.freeze(obj);
}
const config = deepFreeze({ db: { host: 'localhost', port: 5432 } });
tryIt('deepFreeze: nested change', () => { config.db.port = 1; });   // -> TypeError
```

`Object.setPrototypeOf(obj, proto)` works, but engines optimise by object **shape** (internal record of which properties exist, in which order), and swapping prototypes later can slow code — use `Object.create` up front.

When not to use it

- Don't `freeze` large **hot-path** data (speed-critical, run very often) "just in case"; freeze exported constants and config, or use TypeScript's `readonly` for compile-time safety (checked before running — Chapter 16).
- Don't `defineProperty` ordinary data; use it for library-style hidden (non-enumerable) helpers, read-only constants, lazy getters (computed on first access).

Common mistakes

- Assuming `Object.freeze` is deep.
- Omitted `defineProperty` flags default to `false` — the property silently becomes hidden and read-only.
- Expecting `Object.keys` to include symbol or non-enumerable keys — use `Reflect.ownKeys` for "everything own".

In the interview

> **30-second answer:** "Every property has a descriptor: value, writable, enumerable, configurable — or get/set for accessors. `preventExtensions` stops additions, `seal` also deletions, `freeze` also writes. All shallow; violations throw only in strict mode."

- *Truly immutable object?* Deep-freeze, or immutable data structures (every update yields a new version — e.g. Immer, persistent collections). `const` does nothing for contents.
- *`Object.keys` vs `Object.getOwnPropertyNames`?* The latter includes non-enumerable string keys (e.g. an array's `length`).
- *`Object.is` vs `===`?* `NaN`/`NaN` (true) and `0`/`-0` (false).

Self-check

1. After `Object.seal(o)`, can you assign an existing `o.x`? → Yes.
2. `Object.defineProperty(o, 'k', { value: 1 })` — does `Object.keys(o)` show `k`? → No (non-enumerable).
3. Which lists symbol keys? → `Object.getOwnPropertySymbols` (or `Reflect.ownKeys`).
4. `Object.isFrozen({})`? → No; `Object.isFrozen(Object.freeze({}))` is.

5.7 Getters and setters

What it is

An **accessor property** looks like a normal property but runs a function when read (**getter**, `get`) or written (**setter**, `set`) — computing on demand or validating assignments without changing how callers use the object.

How it works

From `code/05_getters_setters_optional.mjs`:

```
const account = {
  owner: 'Sam',
  _balance: 100,                    // underscore = "please treat as internal" (convention only)
  get balance() { return `$${this._balance.toFixed(2)}`; },
  set balance(v) {
    if (typeof v !== 'number' || v < 0) throw new RangeError('balance must be a non-negative number');
    this._balance = v;
  },
  deposit(amount) { this._balance += amount; return this; }, // returning this enables chaining
};

console.log(account.balance);           // getter runs; no parentheses
account.balance = 250;                  // setter runs
console.log(account.deposit(10).deposit(5).balance);
try { account.balance = -1; } catch (e) { console.log(e.name + ':', e.message); }
console.log(Object.getOwnPropertyDescriptor(account, 'balance'));

const rect = { w: 3, h: 4, get area() { return this.w * this.h; } };
rect.w = 10;
console.log(rect.area);
```

Output:

```
$100.00
$265.00
RangeError: balance must be a non-negative number
{
  get: [Function: get balance],
  set: [Function: set balance],
  enumerable: true,
  configurable: true
}
40
```

- `account.balance` (no parentheses) calls the getter; assignment calls the setter, which validates and stores into the **backing** property `_balance`.
- The descriptor has `get` / `set` instead of `value` / `writable`.
- `rect.area` is **derived** — always current, never stored.
- Getter and backing property **must have different names** — `this.balance = v` inside the setter would recurse forever. The underscore is only convention; true privacy uses closures (Chapter 4) or `#private` fields (Chapter 9).

When not to use it

Readers expect property reads to be instant and side-effect free — slow work (network, heavy computation) belongs in a method like `loadBalance()`. A getter without a setter makes the property read-only (writes throw in strict mode).

Real-world examples

Vue 2's **reactivity** (screen updates automatically when data changes) was built on `defineProperty` accessors; MobX and many ORMs (Object-Relational Mappers, mapping database rows to objects) use them for computed fields; DOM properties like `innerHTML` and `input.value` are accessors on prototypes.

5.8 Optional chaining `?.`

The problem it solves

API data is often incomplete: `res.data.user.name` throws `TypeError: Cannot read properties of undefined` if any link is missing. Pre-ES2020: `res && res.data && res.data.user && res.data.user.name`.

How it works

`a?.b`: if `a` is `null`/`undefined`, stop and return `undefined`; else read `a.b`. Forms: `obj?.prop`, `obj?.[expr]`, `fn?.(args)`; the rest of the chain is skipped once it short-circuits.

```
const res = { data: { user: { name: 'Ana', tags: ['a'] } } };
const empty = {};
console.log(res.data?.user?.name, empty.data?.user?.name);
console.log(res.data?.user?.tags?.[0], empty.data?.user?.tags?.[0]);   // with brackets
console.log(res.logout?.(), typeof empty.save?.());                    // calling a maybe-missing method
console.log(empty.data?.user?.name ?? 'Guest');                        // pair with ?? for a default
```

Output:

```
Ana undefined
a undefined
undefined undefined
Guest
```

Pair with `??` (Chapter 2) for a default only on `null`/`undefined` (`||` also replaces `0`, `''`, `false`).

Common mistakes

- `?.` everywhere — if a value *must* exist, fail loudly instead of hiding a bug.
- `obj?.x = 1` is a `SyntaxError`.
- `?.` guards only `null`/`undefined`: `(5)?.toFixed` works, `''?.length` is `0`.

5.9 JavaScript's built-in (native) objects

Built-in objects exist in every environment (browser, Node, Deno, Bun) without import. Some are **constructors** (called with `new` or as converters), some **namespaces** (plain objects of static functions):

Built-in	Kind	Used for	Where in this book
`Object`	constructor + static helpers	Base of all objects; `Object.keys`, etc.	this chapter
`String`, `Number`, `Boolean`, `BigInt`, `Symbol`	wrappers / converters	Methods on primitives; conversions	Ch 2, Ch 6
`Array`	constructor	Ordered lists	Ch 7
`Function`	constructor	Every function is an object of this type	Ch 4
`Date`	constructor	Points in time	5.11
`RegExp`	constructor	Pattern matching on text	Ch 6
`Map`, `Set`, `WeakMap`, `WeakSet`, `WeakRef`	constructors	Keyed collections, unique values	Ch 8
`Promise`	constructor	Results that arrive later	Ch 12
`Error` (+ `TypeError`, `RangeError`, …)	constructors	Exceptions (error objects thrown to stop normal flow)	Ch 11
`Math`	namespace	Mathematical constants and functions	5.10
`JSON`	namespace	Converting to/from JSON text	5.12
`Intl`	namespace	Locale-aware (language + region, e.g. `en-US`) formatting (dates, numbers, lists)	5.11, Ch 6
`Reflect`, `Proxy`	namespace / constructor	Meta-programming: `Reflect` exposes object operations as functions (`Reflect.ownKeys`, 5.2); a `Proxy` wraps an object and intercepts reads and writes (how Vue 3 reactivity and Immer work)	overview only
`ArrayBuffer`, typed arrays	constructors	Raw binary data	Ch 7
`globalThis`	the global object	Access to globals in any environment	Ch 1

`document`, `window`, `fetch`, Node's `process` are **not** language built-ins — they come from the host (browser or Node; Chapters 13–15).

5.10 `Math`

What it is

`Math` is a **namespace object** of constants and functions — not a constructor (`new Math()` throws `TypeError`); call `Math.floor(x)` directly. All members are **static**.

Constants

Property	Value (approx.)	Meaning
`Math.PI`	3.14159	Circle circumference ÷ diameter
`Math.E`	2.71828	Euler's number, base of natural logarithms
`Math.SQRT2` / `Math.SQRT1_2`	1.41421 / 0.70711	√2 and √½
`Math.LN2` / `Math.LN10`	0.69315 / 2.30259	Natural log of 2 and of 10
`Math.LOG2E` / `Math.LOG10E`	1.44270 / 0.43429	$\log_2(e)$ and $\log_{10}(e)$

Methods by family

Family	Methods	Notes
Rounding	`round`, `floor`, `ceil`, `trunc`, `fround`	See the table below for negatives
Sign & size	`abs`, `sign`	`sign` returns -1, 0, 1 (or -0/NaN)
Extremes	`max(...xs)`, `min(...xs)`	Take separate arguments — spread arrays in
Powers & roots	`pow(x, y)` (same as `x ** y`), `sqrt`, `cbrt`, `hypot(...xs)` (√ of sum of squares)	
Exp & logs	`exp`, `expm1`, `log` (natural), `log10`, `log2`, `log1p`	`log(0)` is `-Infinity`; negative → `NaN`
Trigonometry (radians — an angle unit where a full circle is 2π)	`sin`, `cos`, `tan`, `asin`, `acos`, `atan`, `atan2(y, x)`, and hyperbolic `sinh`, `cosh`, `tanh`, …	`atan2` gives the angle of point (x, y), full −π…π range
Random	`random()`	Float in **`[0, 1)`** — 0 possible, 1 never
Integer tricks	`clz32`, `imul`	Low-level 32-bit helpers

Newer: `Math.f16round` (ES2025, rounds to half-precision float; in Node 24) and `Math.sumPrecise` (ES2026, sums without accumulated rounding error — not in Node 24; check support).

How it works — examples

From `code/06_math.mjs`:

```
const r = 5;
console.log('area', Math.PI * r ** 2, Math.PI * Math.pow(r, 2));

const rows = [3.56, 3.5, -3.5, -3.56, 2.4].map((x) => ({
  x, round: Math.round(x), floor: Math.floor(x), ceil: Math.ceil(x), trunc: Math.trunc(x),
}));
console.table(rows);

console.log(Math.round(1.2345 * 100) / 100, (1.005).toFixed(2), (1.2345).toFixed(2));

const temps = [18, 25, 21, 30, 17];
console.log(Math.max(10, 20, 30, 40, 50), Math.min(...temps), Math.max(...temps));
console.log(Math.max(), Math.min(), Math.max(1, 'x'));

const clamp = (v, lo, hi) => Math.min(Math.max(v, lo), hi);
console.log(clamp(120, 0, 100), clamp(-5, 0, 100), clamp(42, 0, 100));
```

Output:

```
area 78.53981633974483 78.53981633974483
┌─────────┬─────────┬─────────┬─────────┬─────────┬─────────┐
│ (index) │ x       │ round   │ floor   │ ceil    │ trunc   │
├─────────┼─────────┼─────────┼─────────┼─────────┼─────────┤
│ 0       │ 3.56    │ 4       │ 3       │ 4       │ 3       │
│ 1       │ 3.5     │ 4       │ 3       │ 4       │ 3       │
│ 2       │ -3.5    │ -3      │ -4      │ -3      │ -3      │
│ 3       │ -3.56   │ -4      │ -4      │ -3      │ -3      │
│ 4       │ 2.4     │ 2       │ 2       │ 3       │ 2       │
└─────────┴─────────┴─────────┴─────────┴─────────┴─────────┘
1.23 1.00 1.23
50 17 30
-Infinity Infinity NaN
100 0 42
```

- `round` **rounds halves toward +∞**: `Math.round(-3.5)` is `-3` (not "half away from zero" as in many languages).
- `floor` **vs** `trunc` differ only for negatives: −3.5 → −4 vs −3.
- `toFixed` **returns a string** and inherits binary-float error: 1.005 is stored as 1.00499999…, so `(1.005).toFixed(2)` is `"1.00"`. For money use integer cents, display with `Intl.NumberFormat` (Chapter 2).
- `Math.max()` with no arguments is `-Infinity` (the identity for max); any argument converting to `NaN` (`'x'`) gives `NaN` (numeric strings like `'5'` work). Spreading a *huge* array can exceed the engine's argument limit — use `reduce` for hundreds of thousands of items.

```js
console.log(Math.abs(-7), Math.sign(-7), Math.sqrt(81), Math.cbrt(27), Math.hypot(3, 4));
console.log(Math.pow(2, 10), 2 ** 10, Math.exp(1), Math.log(Math.E), Math.log10(1000), Math.log2(8));

const toRad = (deg) => (deg * Math.PI) / 180;
console.log(Math.sin(toRad(30)).toFixed(4), Math.cos(toRad(60)).toFixed(4), Math.sin(Math.PI));
console.log('angle of (1,1):', (Math.atan2(1, 1) * 180) / Math.PI, 'deg');
```

Output:

```
7 -1 9 3 5
1024 1024 2.718281828459045 1 3 3
0.5000 0.5000 1.2246467991473532e-16
angle of (1,1): 45 deg
```

Trig uses **radians** (2π = 360°); `Math.sin(Math.PI)` isn't exactly 0 because π can't be stored exactly.

Random integers

`Math.random()` gives `0 ≤ x < 1`. For an integer in `[min, max]` **inclusive**: multiply by the count `max - min + 1` (float in `[0, count)`), `Math.floor` (integer `0 … count-1`), add `min` (integer `min … max`):

```js
const randInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;
```

The demo draws 30,000 values from `randInt(1, 3)`; all land in `{1, 2, 3}` roughly evenly. Using `Math.round` instead of `floor` is a classic bug: the end values get half the probability.

`Math.random` **is not cryptographically secure** (its output can be predicted by attackers). For tokens, IDs, passwords use the Web Crypto API: `crypto.randomUUID()` (a **UUID** is a random 128-bit, practically unique identifier) or `crypto.getRandomValues(new Uint32Array(1))` (global in browsers and Node).

In the interview

- *`0.1 + 0.2 !== 0.3`; round to two decimals?* IEEE-754 binary floats can't store 0.1 exactly. Compare with a tolerance (`Math.abs(a - b) < Number.EPSILON`), round with `Math.round(x * 100) / 100`, or use cents.
- *Random integer in [a, b]?* `Math.floor(Math.random() * (b - a + 1)) + a`.
- *Max of an array?* `Math.max(...arr)`; `arr.reduce((m, x) => x > m ? x : m)` for huge arrays.

Self-check

1. `Math.round(-2.5)` ? → `-2`.
2. `Math.trunc(-4.7)` vs `Math.floor(-4.7)` ? → `-4` vs `-5`.
3. Can `Math.random()` return 1? → No.
4. 180° in radians? → `Math.PI`.

5.11 `Date`

What it is

A `Date` is **one instant in time**, stored as milliseconds since the **Unix epoch**, 1 January 1970 00:00:00 **UTC** (Coordinated Universal Time). Year, month, hour, weekday are *computed* from that number in **local time** (the running machine's zone) or UTC. A `Date` has no time zone of its own — the zone appears only when you read or print it.

Why it matters

Calendars, bookings, logs, subscriptions, "3 hours ago" all need time. `Date` is infamous for pitfalls — 0-based months, mutability, parsing quirks, DST (daylight-saving time, clocks jumping an hour) — hence the new **Temporal** API (end of section).

Creating dates

Form	Creates	Example
`new Date()`	The current instant	`new Date()`
`new Date(ms)`	Instant `ms` milliseconds after the epoch (negative = before 1970)	`new Date(0)` → 1970-01-01T00:00:00Z
`new Date(string)`	Parsed from text; reliable only for **ISO 8601** (`YYYY-MM-DDTHH:mm:ss.sssZ`)	`new Date('2024-03-10T14:30:00Z')`
`new Date(y, m, d?, h?, min?, s?, ms?)`	Built from **local-time** parts; **month is 0-based** (0 = January, 11 = December); omitted day = 1, omitted time parts = 0	`new Date(2024, 0, 31, 9, 15)`
`Date.now()`	Not a Date — just the current ms number	handy for timing
`Date.UTC(y, m, …)`	ms number for those parts **in UTC**	`new Date(Date.UTC(2024, 0, 1))`

`Date()` *without* `new` returns a string — always use `new`.

From `code/07_date.mjs` (it pins `process.env.TZ = 'America/New_York'` — `process.env` holds environment variables — for repeatable output):

```
const now = new Date();                                  // current moment
console.log('now is a Date:', now instanceof Date, typeof now.getTime());
const epoch = new Date(0);                               // ms since 1970-01-01T00:00:00Z
const fromMs = new Date(1_700_000_000_000);
const fromIso = new Date('2024-03-10T14:30:00Z');        // ISO 8601 string (Z = UTC)
const fromParts = new Date(2024, 0, 31, 9, 15);          // local time; month is 0-based (0 = Jan)
console.log(epoch.toISOString(), fromMs.toISOString());
console.log(fromIso.toISOString(), fromParts.toString());
```

Output:

```
now is a Date: true number
1970-01-01T00:00:00.000Z 2023-11-14T22:13:20.000Z
2024-03-10T14:30:00.000Z Wed Jan 31 2024 09:15:00 GMT-0500 (Eastern Standard Time)
```

Parsing pitfalls

```
console.log(new Date('2023-01-01').toString());
console.log(new Date('2023-01-01T00:00').toString());
console.log(new Date('not a date').getTime(), String(new Date('nope')));
```

Output:

```
Sat Dec 31 2022 19:00:00 GMT-0500 (Eastern Standard Time)
Sun Jan 01 2023 00:00:00 GMT-0500 (Eastern Standard Time)
NaN Invalid Date
```

- A **date-only** ISO string is **UTC midnight** (still 31 December in New York); a date-*time* string without `Z`/offset is **local** — a historical inconsistency in the spec.
- Non-ISO strings (`'March 25, 2015'`, `'03/25/2015'`) are parsed implementation-specifically (each engine may differ) — prefer ISO or numeric parts.
- Unparseable input gives an **Invalid Date** (time value `NaN`): test `Number.isNaN(d.getTime())`.

Reading components — getters

Local getter	UTC twin	Returns
`getFullYear()`	`getUTCFullYear()`	4-digit year (never use the obsolete `getYear`)
`getMonth()`	`getUTCMonth()`	**0–11**
`getDate()`	`getUTCDate()`	Day of month **1–31**
`getDay()`	`getUTCDay()`	Day of week **0–6**, 0 = Sunday
`getHours()` / `getMinutes()` / `getSeconds()` / `getMilliseconds()`	`getUTC…`	0–23 / 0–59 / 0–59 / 0–999
`getTime()` / `valueOf()`	—	ms since the epoch (same instant in any zone)
`getTimezoneOffset()`	—	Minutes to add to local time to get UTC (positive west of Greenwich, e.g. 240 for New York in summer)

```
const d = new Date('2024-07-04T23:30:00Z');
console.log('local:', d.getFullYear(), d.getMonth(), d.getDate(), d.getDay(), d.getHours(), d.getMinutes());
console.log('UTC  :', d.getUTCFullYear(), d.getUTCMonth(), d.getUTCDate(), d.getUTCDay(), d.getUTCHours());
console.log('offset minutes (UTC - local):', d.getTimezoneOffset());
```

Output:

```
local: 2024 6 4 4 19 30
UTC  : 2024 6 4 4 23
offset minutes (UTC - local): 240
```

Month `6` = **July**, weekday `4` = **Thursday**; the instant is 19:30 in New York, 23:30 UTC.

Changing components — setters

Setters (`setFullYear`, `setMonth`, `setDate`, `setHours`, `setMinutes`, `setSeconds`, `setMilliseconds`, `setTime`, plus `setUTC…`) **mutate** in place. Some take several parts (`setFullYear(year, month?, day?)`, `setHours(h, m?, s?, ms?)`); out-of-range values **roll over** (Jan 32 → Feb 1; day 0 → last day of previous month).

```
const b = new Date(2024, 5, 15);
b.setFullYear(1990, 0, 1);                  // year, month, day in one call
console.log(b.toDateString());
const roll = new Date(2024, 0, 31);
roll.setMonth(1);                           // Feb 31 does not exist -> Mar 2 (2024 is a leap year)
console.log(roll.toDateString());
const addDays = (date, n) => { const c = new Date(date); c.setDate(c.getDate() + n); return c; };
console.log(addDays(new Date(2024, 11, 30), 5).toDateString());
const lastDayOfFeb = new Date(2024, 2, 0).getDate();   // day 0 of March = last day of Feb
console.log('days in Feb 2024:', lastDayOfFeb);
```

Output:

```
Mon Jan 01 1990
Sat Mar 02 2024
Sat Jan 04 2025
days in Feb 2024: 29
```

`addDays` copies first so the caller's Date isn't mutated, and rollover crosses the year. "Month-end overflow" (`Jan 31 + 1 month = Mar 2`) is a real billing bug — decide explicitly if you want the month's last day.

Differences and comparisons

Subtracting Dates converts both to ms (via `valueOf`).

```
const start = new Date('2023-01-01');
const end = new Date('2023-12-31');
const MS_PER_DAY = 1000 * 60 * 60 * 24;
console.log((end - start) / MS_PER_DAY + ' days');
const beforeDst = new Date(2024, 2, 9), afterDst = new Date(2024, 2, 11);
console.log((afterDst - beforeDst) / MS_PER_DAY, Math.round((afterDst - beforeDst) / MS_PER_DAY));

const x = new Date(2024, 0, 1), y = new Date(2024, 0, 1);
console.log(x === y, x.getTime() === y.getTime(), x < addDays(y, 1));
```

Output:

```
364 days
1.9583333333333333 2
false true true
```

- 1 Jan → 31 Dec is 364 days (add 1 to count both ends).
- Across the March DST switch two local midnights are 47 hours apart (1.958 "days") — round, or compute in UTC.
- `<` / `>` compare the numbers; `===` compares identity — compare `getTime()` for equality.

Formatting

Method	Output (for the instant 2024-03-10 14:30 UTC, local zone New York)	Use for
`toISOString()`	`2024-03-10T14:30:00.000Z`	Storage, APIs, logs — always UTC, sortable
`toJSON()`	same as ISO	Used automatically by `JSON.stringify`
`toString()`	`Sun Mar 10 2024 10:30:00 GMT-0400 (Eastern Daylight Time)`	Debugging
`toDateString()` / `toTimeString()`	`Sun Mar 10 2024` / `10:30:00 GMT-0400 (…)`	Debugging
`toUTCString()`	`Sun, 10 Mar 2024 14:30:00 GMT`	HTTP headers (name/value lines sent with a request or response)
`toLocaleDateString(locale, opts)` / `toLocaleTimeString` / `toLocaleString`	`3/10/2024` (en-US), `10/03/2024` (en-GB), `10.3.2024` (de-DE)	Showing dates to people
`Intl.DateTimeFormat(locale, opts).format(d)`	reusable formatter	Formatting many dates
`Intl.RelativeTimeFormat`	`yesterday`, `in 3 weeks`	"time ago" labels

```
console.log(t.toLocaleString('en-US', { timeZone: 'Asia/Kolkata', dateStyle: 'full', timeStyle: 'short' }));
const fmt = new Intl.DateTimeFormat('en', { month: 'short', day: 'numeric', year: 'numeric', timeZone: 'UTC' });
console.log(fmt.format(t));
const rtf = new Intl.RelativeTimeFormat('en', { numeric: 'auto' });
console.log(rtf.format(-1, 'day'), '|', rtf.format(3, 'week'));
```

Output:

```
Sunday, March 10, 2024 at 8:00 PM
Mar 10, 2024
yesterday | in 3 weeks
```

`timeZone` *displays* an instant in any IANA zone (official time-zone database names: `'Asia/Kolkata'`, `'Europe/Berlin'`), but `Date` can't *do arithmetic* in a chosen zone — the gap Temporal fills.

Temporal — the modern replacement (status in 2026)

Temporal is a new global namespace of immutable date/time types. Verified status (September 2026):

- Reached **Stage 4** at the TC39 meeting of **March 2026**; TC39's finished-proposals list places it in **ES2027** (some articles say "ES2026"; that snapshot was already cut).
- Shipped by default in **Firefox 139+**, **Chrome/Edge 144+** and **Node.js 26+**. **Safari** stable hasn't shipped it yet, and **Node 24** (this book's runtime) lacks it — our script prints `Temporal available here: false`. Use `@js-temporal/polyfill` or `temporal-polyfill` (a **polyfill** adds a missing built-in to older environments).

Type	Represents	Example
`Temporal.Instant`	An exact moment (like a Date's number, in nanoseconds)	a log timestamp
`Temporal.ZonedDateTime`	Instant + time zone + calendar; DST-aware arithmetic	a meeting in Berlin
`Temporal.PlainDate` / `PlainTime` / `PlainDateTime`	Wall-clock values with **no** time zone	a birthday; a store's opening time
`Temporal.PlainYearMonth` / `PlainMonthDay`	Partial dates	a credit-card expiry; an anniversary
`Temporal.Duration`	A length of time	"2 hours 30 minutes"
`Temporal.Now`	Current time in various forms	`Temporal.Now.plainDateISO()`

A taste (Node 26 / Chrome 144 / Firefox 139 — not run on Node 24):

```
const due = Temporal.PlainDate.from('2024-01-31').add({ months: 1 });
due.toString();                                  // '2024-02-29' — clamps to month end, months are 1-based
const meeting = Temporal.ZonedDateTime.from('2026-03-28T09:00[Europe/Berlin]');
meeting.add({ days: 1 }).toString();             // DST-aware: still 09:00 local the next day
```

Better because: **immutable** values, **1-based** months, strict ISO parsing, first-class time zones and calendars, and separate types for "instant" vs "wall-calendar date". Until it's everywhere you target, use **date-fns**, **Luxon** or **Day.js**.

Common mistakes

- 0-based months (`new Date(2024, 1, 1)` is **February** 1).
- `getDate()` (day of month) vs `getDay()` (weekday).
- `'2023-01-01'` showing 31 December in the Americas.
- Mutating a shared Date — copy with `new Date(d)` first.
- Storing local-time strings — store UTC (ISO or epoch ms), convert for display.
- "Days between" by raw division across DST.

Real-world examples

- APIs send ISO 8601 (`"2024-05-01T08:00:00Z"`); clients parse with `new Date(str)` and display with `Intl.DateTimeFormat`.
- Timing uses `Date.now()` or better `performance.now()` — **monotonic** (never goes backward, even if the clock changes) and sub-millisecond.
- JWTs (JSON Web Tokens — signed login tokens) store expiry in **seconds** since the epoch — multiply by 1000 for `new Date`.

In the interview

> **30-second answer:** "A Date wraps one number — ms since 1970 UTC; getters compute local or UTC parts. Months are 0-based, setters mutate and roll over, date-only ISO strings parse as UTC. I display with `Intl.DateTimeFormat`; for serious date math I use Temporal (standard, shipping in Chrome, Firefox and Node 26) or a library."

- *Days in a month?* `new Date(year, month + 1, 0).getDate()`.
- *Valid Date?* `!Number.isNaN(d.getTime())`.
- *Why does `new Date('2024-01-01')` show the wrong day?* Parsed as UTC midnight, shown in a zone west of UTC.

Self-check

1. `new Date(2025, 11, 25).getMonth()`? → `11`.

2. `getDay()` for Sunday? → `0`.
3. `new Date(2024, 0, 32)`? → 1 February 2024.
4. ms in a day? → 86,400,000 (ignoring DST days).
5. Store which format? → UTC — ISO 8601 or epoch ms.

5.12 JSON

What it is

JSON (JavaScript Object Notation) is a *text* format for structured data, standardised as ECMA-404 / RFC 8259 (an RFC is a numbered internet standards document). It borrows JS literal syntax but is stricter, and every language and database speaks it.

- `JSON.stringify(value, replacer?, space?)` → JSON **string** (serialisation).
- `JSON.parse(text, reviver?)` → values (deserialisation).

JSON syntax rules (vs JS literals)

Allowed in JSON	Not allowed in JSON (but fine in JS)
Objects `{}` with **double-quoted** keys	Unquoted or single-quoted keys: `{a: 1}`, `{'a': 1}`
Strings in **double quotes**	Single quotes, template literals
Numbers (decimal), `true`, `false`, `null`	`undefined`, `NaN`, `Infinity`, hex `0xFF`, BigInt
Arrays `[]`	Trailing commas `[1, 2,]`
Nesting to any depth	Comments, functions, Dates, Maps, Sets, symbols

How `stringify` works

From `code/08_json.mjs`:

```
const order = {
  id: 42, customer: 'Ravi', total: 19.99, paid: true, coupon: null,
  items: [{ sku: 'A1', qty: 2 }],
  placedAt: new Date('2024-05-01T08:00:00Z'),
};

const text = JSON.stringify(order);
console.log(text);
console.log(typeof text, text.length);
console.log(JSON.stringify({ a: 1, b: [1, 2] }, null, 2));
console.log(JSON.stringify(order, ['id', 'total']));
console.log(JSON.stringify(order, (key, value) => (key === 'customer' ? undefined : value)).slice(0, 60) + '...');
```

Output:

```
{"id":42,"customer":"Ravi","total":19.99,"paid":true,"coupon":null,"items":
[{"sku":"A1","qty":2}],"placedAt":"2024-05-01T08:00:00.000Z"}
string 136
{
  "a": 1,
  "b": [
    1,
    2
  ]
}
{"id":42,"total":19.99}
{"id":42,"total":19.99,"paid":true,"coupon":null,"items":[{"...
```

1. The result is a **string**; the Date became ISO via `Date.prototype.toJSON`.
2. `space` (up to 10 spaces, or a string like `'\t'`) pretty-prints.

3. The **replacer** is an **array** of allowed keys or a **function** `(key, value) => newValue` per property (`undefined` drops it).

What JSON silently changes or refuses

```
const weird = {
  f() {}, u: undefined, s: Symbol('x'), nan: NaN, inf: Infinity, negZero: -0,
  map: new Map([['k', 1]]), set: new Set([1]), arr: [undefined, () => 1, NaN],
};
console.log(JSON.stringify(weird));
try { JSON.stringify({ big: 10n }); } catch (e) { console.log('BigInt ->', e.constructor.name); }
const loop = { name: 'me' }; loop.self = loop;
try { JSON.stringify(loop); } catch (e) { console.log('circular ->', e.constructor.name); }

const money = { cents: 1999, toJSON() { return (this.cents / 100).toFixed(2); } };
console.log(JSON.stringify({ price: money }));
```

Output:

```
{"nan":null,"inf":null,"negZero":0,"map":{},"set":{},"arr":[null,null,null]}
BigInt -> TypeError
circular -> TypeError
{"price":"19.99"}
```

Value	In an object property	In an array	At top level
`undefined`, function, symbol	**property dropped**	becomes `null`	`JSON.stringify` returns `undefined`
`NaN`, `Infinity`	`null`	`null`	`"null"`
`-0`	`0`	`0`	`"0"`
`Date`	ISO string (via `toJSON`)	same	same
`Map`, `Set`	`{}` — data lost!	same	same
BigInt	**throws** `TypeError`	throws	throws
Circular reference	**throws** `TypeError`	throws	—
Non-enumerable or symbol keys	skipped	—	—

Any object can define **`toJSON()`** to control its representation. Convert a `Map` first: `Object.fromEntries(map)` or `[...map]`.

How `parse` works

```
const parsed = JSON.parse(text);
console.log(parsed.items[0].qty, typeof parsed.placedAt);          // dates come back as strings
const revived = JSON.parse(text, (key, value) => (key === 'placedAt' ? new Date(value) : value));
console.log(revived.placedAt instanceof Date, revived.placedAt.getUTCHours());

for (const bad of ["{'a': 1}", '{a: 1}', '{"a": 1,}', 'undefined', '']) {
  try { JSON.parse(bad); console.log('ok', bad); } catch (e) { console.log('SyntaxError for',
JSON.stringify(bad)); }
}
console.log(JSON.parse('"just a string"'), JSON.parse('3.5'), JSON.parse('[true,null]'));
console.log(JSON.parse('{"id": 9007199254740993}').id);
```

Output:

```
2 string
true 8
SyntaxError for "{'a': 1}"
SyntaxError for "{a: 1}"
SyntaxError for "{\"a\": 1,}"
SyntaxError for "undefined"
SyntaxError for ""
just a string 3.5 [ true, null ]
9007199254740992
```

- No date type: dates return as **strings**; a **reviver** `(key, value) => newValue` runs on every value (innermost first) to restore them.
- Invalid JSON throws `SyntaxError` — wrap `JSON.parse` of external input in `try/catch`.
- Any JSON value can be top level.
- Integers above `Number.MAX_SAFE_INTEGER` ($2^{53} - 1$) lose precision (`…993` → `…992`) — why APIs like Twitter/X send big IDs as strings. ES2026's **"JSON.parse source text access"** passes the reviver a third argument whose `source` is the original text, so you can rebuild an exact BigInt (Node 24 supports it):

```
const exact = JSON.parse('{"id": 9007199254740993}', (key, value, ctx) =>
  key === 'id' ? BigInt(ctx.source) : value);
console.log(exact.id);   // 9007199254740993n
```

Deep copy via JSON vs `structuredClone`

```
const src = { when: new Date(0), tags: new Set(['x']), n: 1 };
console.log(JSON.parse(JSON.stringify(src)), structuredClone(src));
```

Output:

```
{ when: '1970-01-01T00:00:00.000Z', tags: {}, n: 1 } { when: 1970-01-01T00:00:00.000Z, tags: Set(1) { 'x' }, n: 1
}
```

JSON turned the Date into a string and emptied the Set; `structuredClone` kept both. Use `structuredClone` for copying, JSON for **transport** — HTTP bodies, files, `localStorage` (the browser's small key-value store), message queues (services holding messages until another program takes them — System Design Fundamentals).

Real-world examples

- `fetch`: `await response.json()` parses the body; send with `body: JSON.stringify(data)` and `Content-Type: application/json` (Chapter 14).
- `package.json` (project settings and dependencies), `tsconfig.json` (TypeScript settings — actually "JSONC", allowing comments) and most config files.
- `localStorage` stores only strings → `JSON.stringify` / `JSON.parse`.
- Log pipelines emit one JSON object per line ("NDJSON") so tools like Elasticsearch (a search engine for logs) can index fields.

In the interview

30-second answer: "`JSON.stringify` serialises to a string, dropping functions/undefined/symbols in objects, turning NaN/Infinity into null and Dates into ISO strings via `toJSON`, and throwing on BigInt and cycles. `JSON.parse` is strict — double quotes, no trailing commas — and takes a reviver to restore types like Dates."

- *Circular structures?* A replacer tracking seen objects in a `WeakSet` (doesn't keep them alive, Chapter 8) to drop/mark repeats, or restructure; `structuredClone` copies cycles but doesn't help sending JSON.
- *`JSON.stringify({a: undefined})` → `{}`?* `undefined` properties are omitted.

- *Is JSON a subset of JavaScript?* Since ES2019 ("JSON superset"), yes.

Self-check

1. `JSON.stringify([undefined])`? → `"[null]"`.
2. `JSON.stringify({ d: new Date(0) })`? → `{"d":"1970-01-01T00:00:00.000Z"}`.
3. `{"a":1,}` valid? → No — trailing comma.
4. Reviver's job? → Transform parsed values (e.g. strings → Dates).

Chapter TL;DR

- **Objects** are key → value collections; literals `{}`; **dot** for fixed identifier keys, **bracket** for dynamic/unusual keys; computed keys `[expr]` and shorthand `{ name }`.
- Key order: integer-like ascending, then strings by insertion, then symbols.
- Variables hold **references**; `===` compares identity; `const` doesn't freeze contents.
- **Spread** and `Object.assign` are **shallow** (later keys win); `structuredClone` deep-copies.
- **Destructuring**: renames (`{ id: orderId }`), defaults (only `undefined`), nesting, parameters.
- `Object.keys/values/entries/fromEntries`; `hasOwn` for own keys; `Object.is` fixes `NaN`/`-0`; `Object.create(proto | null)` sets the prototype.
- **Descriptors** (`writable`, `enumerable`, `configurable`, `get`/`set`); `preventExtensions` < `seal` < `freeze`, all shallow.
- **Getters/setters** compute or validate on access; keep them cheap.
- `?.` short-circuits on `null`/`undefined`; pair with `??`.
- `Math`: `round` (halves toward +∞), `floor`/`ceil`/`trunc`, `max/min(...arr)`, radians for trig; random int = `Math.floor(Math.random() * (max - min + 1)) + min`; `crypto` for security.
- `Date` = ms since 1970 UTC; months 0-based, `getDay` = weekday, setters mutate and roll over, date-only ISO parses as UTC; format with `Intl`. **Temporal**: Stage 4 (March 2026), shipping in Chrome, Firefox, Node 26 — not Node 24/Safari.
- `JSON.stringify` drops/changes non-JSON values, throws on BigInt/cycles; `JSON.parse` is strict and takes a reviver.